Object Oriented Programming(C++)
(CSE 1211)

Abstraction

Md. Mahfuzur Rahman
Lecturer,
Prime University

Abstraction

- **Abstraction** is the process of hiding implementation details and showing only the functionally to the user. Such as: ATM machine, message, calculator etc.
- **Data abstraction** is one of the most essential and important features of object-oriented programming in C++. Abstraction means displaying only essential information about the data and ignoring the internal details or implementation.
- Consider a ***real-life example of a man driving a car***. The man only knows that pressing the accelerator will increase the speed of the car or applying brakes will stop the car but he does not know how on pressing the accelerator the speed is actually increasing, he does not know about the inner mechanism of the car or the implementation of the accelerator, brakes, etc in the car.

Abstraction

- An **abstract class** is a class that cannot be used to create objects directly. It is meant to be a **base class** that provides a common interface but **doesn't define full behavior**.
- A class becomes abstract if it has at least one pure virtual function.

Pure Virtual Function

A **pure virtual function** is a function declared in a base class that **must be overridden** in any derived class.

```
virtual void draw() = 0;  // Pure virtual function
```

The = 0 means “no default implementation”.

It forces derived classes to implement the function.

Abstract Classes and Pure Virtual Functions

Abstraction in C++ is a fundamental concept in object-oriented programming (OOP) that focuses on simplifying complex systems by hiding unnecessary details and showing only the essential features to the user. It allows developers to create a high-level view of functionality without revealing the underlying Implementation.

Abstract Classes and Pure Virtual Functions:

An abstract class is a class that cannot be instantiated directly and serves as a base for other classes.

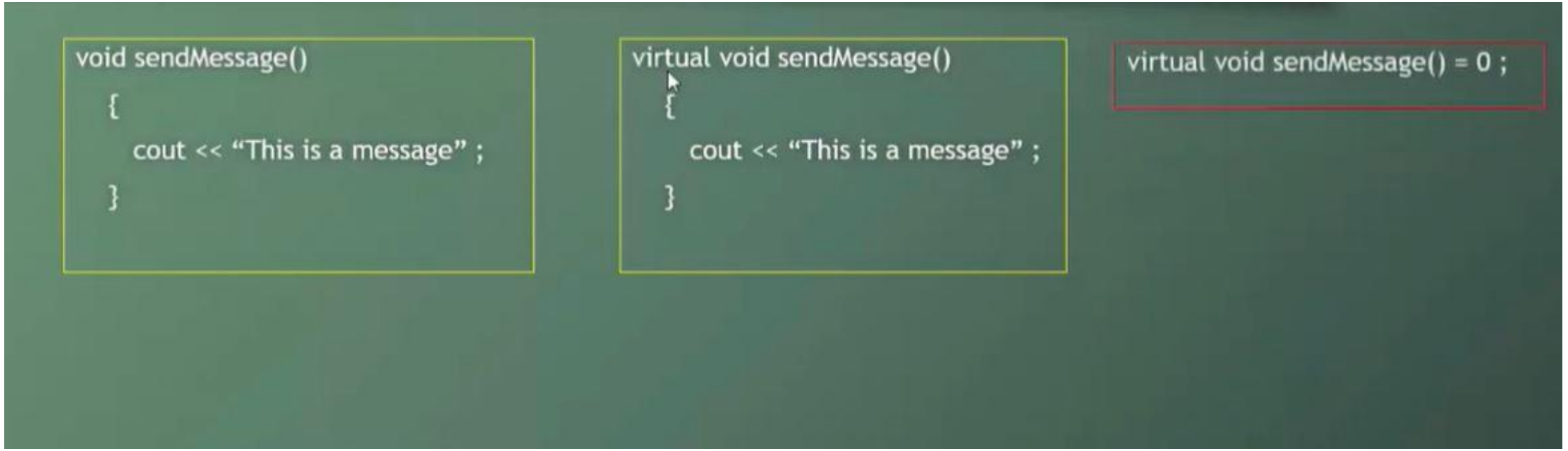
It contains at least one pure virtual function, indicated by = 0 in its declaration.

Pure virtual functions declare an interface that derived classes must implement, enforcing a common behavior without providing a default implementation in the base class.

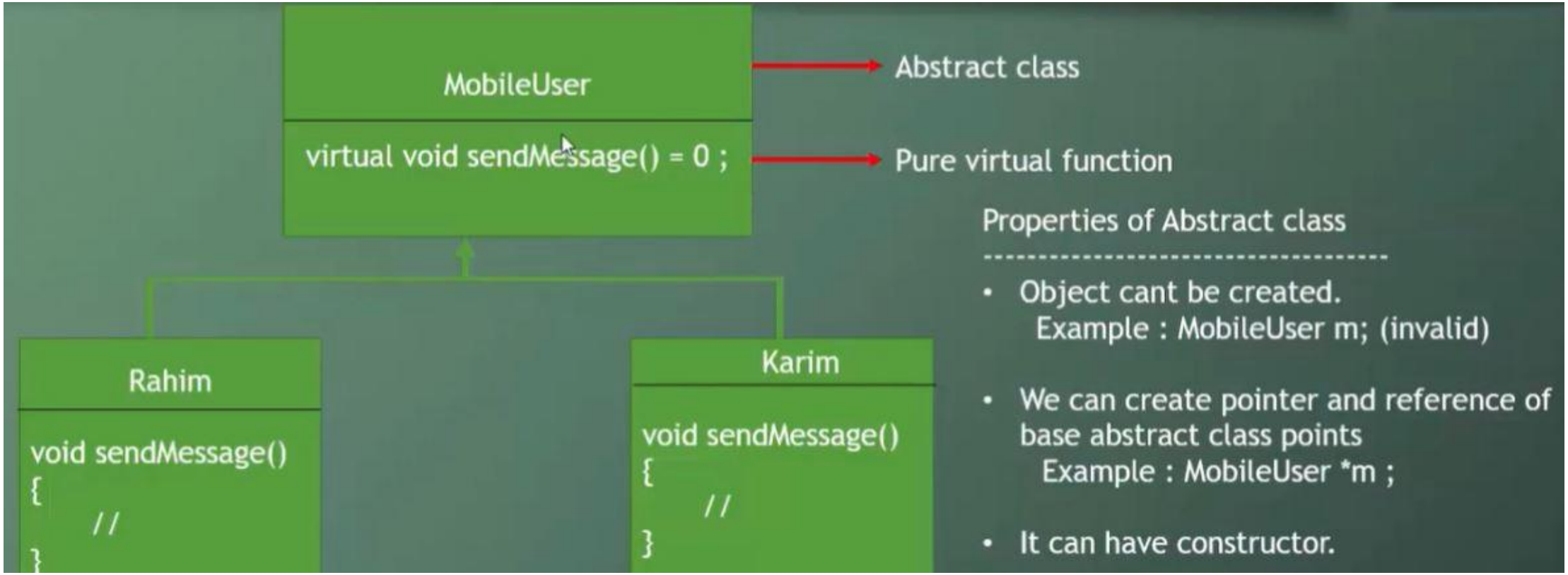
Benefits of Abstraction

- **Simplifies Complexity:** Users interact with a simplified interface, reducing cognitive load.
- **Enhances Security:** Hides sensitive data and implementation details, preventing direct manipulation.
- **Improves Maintainability:** Changes to internal implementation details can be made without affecting external code that uses the abstracted interface.
- **Promotes Reusability:** Abstract classes and interfaces provide a blueprint for consistent and reusable components.

Abstraction



Abstraction



Abstraction

```cpp
#include<iostream>
using namespace std;

class MobileUser
{
public:
    void call()
    {
        cout << "Hello" << endl;
    }
    virtual void sendMessage() = 0;
};

class Rahim: public MobileUser
{
public:
    void sendMessage()
    {
        cout << "This is Rahim"<< endl;
    }
};

class Karim: public MobileUser
{
public:
    void sendMessage()
    {
        cout << "This is Karim"<< endl;
    }
};
```

```cpp
int main()
{
    MobileUser *m;

    Rahim r;
    Karim k;

    m = &r;
    m->sendMessage();

    m->call();

    m = &k;
    m->sendMessage();
}
```

```
"E:\Prime University\02. Summer_25
This is Rahim
Hello
This is Karim

Process returned 0 (0x0)
Press any key to continue.
```

Abstraction

- Data abstraction is one of the most essential and important features of object-oriented programming in C++. Abstraction means displaying only essential information about the data and ignoring the internal details or implementation.
- Consider a ***real-life example of a man driving a car***. The man only knows that pressing the accelerator will increase the speed of the car or applying brakes will stop the car but he does not know how on pressing the accelerator the speed is actually increasing, he does not know about the inner mechanism of the car or the implementation of the accelerator, brakes, etc in the car. This is what abstraction is.

Types of Abstraction:

- **Data abstraction -** This type only shows the required information about the data and ignores unnecessary details.
- **Control Abstraction -** This type only shows the required information about the implementation and ignores unnecessary details.

Abstraction

Abstraction in C++ is the process to hide the internal details and showing functionality only. Abstraction can be achieved by two ways:

1. **Abstract class**
2. **Interface**

Abstract class and interface both can have abstract methods which are necessary for abstraction.

C++ Abstract class:

In C++ class is made abstract by declaring at least one of its functions as <>strong>pure virtual function. A pure virtual function is specified by placing "= 0" in its declaration. Its implementation must be provided by derived classes.

```
#include <iostream>
using namespace std;
 class Shape
{
    public:
    virtual void draw()=0;
};
 class Rectangle : Shape
{
    public:
     void draw()
    {
        cout < <"drawing rectangle..." < <endl;
    }
};
class Circle : Shape
{
    public:
     void draw()
    {
        cout <<"drawing circle..." < <endl;
    }
};
int main( ) {
    Rectangle rec;
    Circle cir;
    rec.draw();
    cir.draw();
   return 0;
}
```

Output:

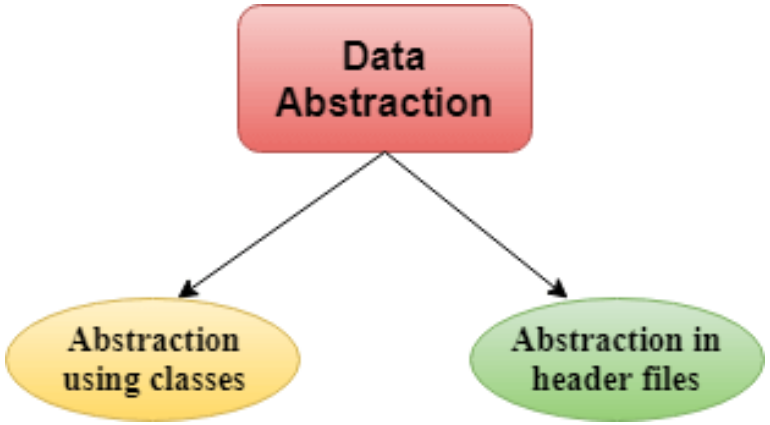
drawing rectangle...
drawing circle...

Data Abstraction in C++

- Data Abstraction is a process of providing only the essential details to the outside world and hiding the internal details, i.e., representing only the essential details in the program.
- Data Abstraction is a programming technique that depends on the separation of the interface and implementation details of the program.
- Let's take a real life example of AC, which can be turned ON or OFF, change the temperature, change the mode, and other external components such as fan, swing. But, we don't know the internal details of the AC, i.e., how it works internally. Thus, we can say that AC seperates the implementation details from the external interface.
- C++ provides a great level of abstraction. For example, pow() function is used to calculate the power of a number without knowing the algorithm the function follows.
- In C++ program if we implement class with private and public members then it is an example of data abstraction.

Data Abstraction can be achieved in two ways:

- Abstraction using classes
- Abstraction in header files.



Abstraction using classes: An abstraction can be achieved using classes. A class is used to group all the data members and member functions into a single unit by using the access specifiers. A class has the responsibility to determine which data member is to be visible outside and which is not.

Abstraction in header files: An another type of abstraction is header file. For example, pow() function available is used to calculate the power of a number without actually knowing which algorithm function uses to calculate the power. Thus, we can say that header files hides all the implementation details from the user.

Abstraction using Access Specifiers

Access specifiers are the main pillar of implementing abstraction in C++. We can use access specifiers to enforce restrictions on class members. For example:

- Members declared as **public** in a class can be accessed from anywhere in the program.
- Members declared as **private** in a class, can be accessed only from within the class. They are not allowed to be accessed from any part of the code outside the class.

We can easily implement abstraction using the above two features provided by access specifiers. Say, the members that define the internal implementation can be marked as private in a class. And the important information needed to be given to the outside world can be marked as public. And these public members can access the private members as they are inside the class. Our **C++ Course** covers how to implement abstraction in your C++ programs, making your code cleaner and more manageable.

Abstraction in Header files

One more type of abstraction in C++ can be header files. For example, consider the pow() method present in math.h header file. Whenever we need to calculate the power of a number, we simply call the function pow() present in the math.h header file and pass the numbers as arguments without knowing the underlying algorithm according to which the function is actually calculating the power of numbers.

Advantages of Data Abstraction

- Avoids code duplication and increases reusability.
- Can change the internal implementation of the class independently without affecting the user.
- Helps to increase the security and reduces complexity and redundancy of an application or program as only important details are provided to the user.

Pure Virtual Functions and Abstract Classes in C++

Abstract class is a class that cannot be used to create objects directly. It is meant to be a **base class** that provides a common interface but **doesn't define full behavior**. A class becomes abstract if it has at least one pure virtual function.

A **pure virtual function** in C++ is a **virtual function** that has no definition in the base class and must be overridden in derived classes. It is declared by assigning = 0 in its declaration.

Pure Virtual Function: A **pure virtual function** is a function declared in a base class that **must be overridden** in any derived class.

```
virtual void draw() = 0;  // Pure virtual function
```

The = 0 means “no default implementation”.

It forces derived classes to implement the function.

Runtime Polymorphism using a Pure Virtual Function

```
#include<iostream>
using namespace std;
class B {
   public:
      virtual void s() = 0;
};

class D:public B {
   public:
      void s() {
         cout << "Virtual Function in Derived class\n";
      }
};
int main() {
   B *b;
   D dobj;
   b = &dobj;
   b->s();
}
```

Abstract Class with a Pure Virtual Function

```cpp
#include <iostream>
using namespace std;
class Shape {
   public:
      virtual void draw() = 0;
};

class Circle: public Shape {
   public:
      void draw() override {
      cout << "Drawing a Circle" << endl;
   }
};

class Rectangle: public Shape {
   public:
      void draw() override {
      cout << "Drawing a Rectangle" << endl;
   }
};
```

```cpp
int main() {
   Shape * shape1;
   Circle circleObj;
   Rectangle rectObj;

   shape1 = & circleObj;
   shape1 -> draw();

   shape1 = & rectObj;
   shape1 -> draw();

   return 0;
}
```

```
#include <iostream>
using namespace std;
 class Shape
{
    public:
    virtual void draw()=0;
};
 class Rectangle : Shape
{
    public:
     void draw()
    {
        cout < <"drawing rectangle..." < <endl;
    }
};
class Circle : Shape
{
    public:
     void draw()
    {
        cout <<"drawing circle..." < <endl;
    }
};
int main( ) {
    Rectangle rec;
    Circle cir;
    rec.draw();
    cir.draw();
   return 0;
}
```

Output:

drawing rectangle...
drawing circle...

```
#include <iostream>
using namespace std;
class Base
{
   public:
   virtual void show() = 0;
};
class Derived : public Base
{
   public:
   void show()
   {
      std::cout << "Derived class is derived from the base class." << std::endl;
   }
};
int main()
{
   Base *bptr;
   //Base b;
   Derived d;
   bptr = &d;
   bptr->show();
   return 0;
}
```

Output:

Derived class is derived from the base class.

In the above example, the base class contains the pure virtual function. Therefore, the base class is an abstract base class. We cannot create the object of the base class.

Pure Virtual Functions and Abstract Classes in C++

```cpp
#include <iostream>
using namespace std;

class Shape {
public:
    virtual void draw() = 0; // Pure virtual function
};

class Circle : public Shape {
public:
    void draw() override {
        cout << "Drawing Circle\n";
    }
};

int main() {
    // Shape s;       Error: Cannot create object of abstract class
    Shape* s = new Circle();  // Pointer to abstract class
    s->draw();                // Output: Drawing Circle
    delete s;
}
```

Abstract Class Vs Interface

Feature	Abstract Class (C++)	Interface (Java/C++)
Methods	Can have some implementation	All methods are pure virtual
Inheritance	Single or multiple inheritance	Simulates multiple inheritance in C++
Use	Common base with partial code	Contract with no implementation
In C++	Abstract class with pure virtuals	All pure virtual functions= Interface
In Java	Use abstract or interface keyword	Separate interface keyword available.

An abstract class defines what derived classes must do, but not how.

A pure virtual function is a function with = 0 and must be overridden.

You can't create objects of abstract classes.

Useful when you want to define a common interface but let derived classes define specific behavior.

Difference Between Abstract Class and Interface

Points	Abstract Class	Interface
Definition	Cannot be instantiated; contains both abstract (without implementation) and concrete methods (with implementation)	Specifies a set of methods a class must implement; methods are abstract by default.
Implementation Method	Can have both implemented and abstract methods.	Methods are abstract by default; Java 8, can have default and static methods.
Inheritance	class can inherit from only one abstract class.	A class can implement multiple interfaces.
Access Modifiers	Methods and properties can have any access modifier (public, protected, private).	Methods and properties are implicitly public.
Variables	Can have member variables (final, non-final, static, non-static).	Variables are implicitly public, static, and final (constants).

Key	Abstract Class	Interface
Supported Methods	Abstract class can have both an abstract as well as concrete methods.	Interface can have only abstract methods. Java 8 onwards, it can have default as well as static methods.
Multiple Inheritance	Multiple Inheritance is not supported.	Interface supports Multiple Inheritance.
Supported Variables	final, non-final, static and non-static variables supported.	Only static and final variables are permitted.
Implementation	Abstract class can implement an interface.	Interface can not implement an interface, it can extend an interface.
Keyword	Abstract class declared using abstract keyword.	Interface is declared using interface keyword.
Inheritance	Abstract class can inherit another class using extends keyword and implement an interface.	Interface can inherit only an inteface.
Inheritance	Abstract class can be inherited using extends keyword.	Interface can only be implemented using implements keyword.
Access	Abstract class can have any type of members like private, public.	Interface can only have public members.

Access Specifiers Implement Abstraction:

- **Public specifier:** When the members are declared as public, members can be accessed anywhere from the program.
- **Private specifier:** When the members are declared as private, members can only be accessed only by the member functions of the class.

Example of abstraction in header files:

```
// program to calculate the power of a number.
#include <iostream>
#include<math.h>
using namespace std;
int main()
{
 int n = 4;
   int power = 3;
   int result = pow(n,power);        // pow(n,power) is the  power function
   std::cout << "Cube of n is : " <<result<< std::endl;
   return 0;
}
```

Output:

Cube of n is : 64

Example of data abstraction using classes:

```
#include <iostream>
using namespace std;
 class Sum
{
private: int x, y, z; // private variables
public:
void add()
{
cout<<"Enter two numbers: ";
cin>>x>>y;
z= x+y;
cout<<"Sum of two number is: "<<z<<endl;
}
};

int main()
{
Sum sm;
sm.add();
return 0;
}
```

Output:

Enter two numbers: 3 6
Sum of two number is: 9

Advantages of Data Abstraction

- Avoids code duplication and increases reusability.
- Can change the internal implementation of the class independently without affecting the user.
- Helps to increase the security and reduces complexity and redundancy of an application or program as only important details are provided to the user.

Advantages Of Abstraction:

- Implementation details of the class are protected from the inadvertent user level errors.
- A programmer does not need to write the low level code.
- Data Abstraction avoids the code duplication, i.e., programmer does not have to undergo the same tasks every time to perform the similar operation.
- The main aim of the data abstraction is to reuse the code and the proper partitioning of the code across the classes.
- Internal implementation can be changed without affecting the user level code.

Thank You